

UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche

Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

Ottimizzazione della memoria nella programmazione dinamica per l'allineamento di sequenze

Laureando
Iacopo Giansanti
Matricola 606248

Relatore
Maurizio Patrignani

Correlatore
Vincenzo Bonifaci

Anno Accademico 2025/2026

Ai miei genitori,
non vi potrò mai ringraziare abbastanza.
Questo è il mio regalo per voi.

A Pasquale e SuperMario, miao.

Introduzione

L'informazione genetica costituisce il principale meccanismo attraverso cui le caratteristiche ereditarie vengono conservate e trasmesse tra generazioni. Con la scoperta della struttura del DNA e, successivamente, con lo sviluppo delle tecniche di sequenziamento, è diventato possibile osservare tale informazione non soltanto come fenomeno biologico, ma anche come una sequenza discreta di simboli.

Dal punto di vista computazionale, infatti, un filamento di DNA può essere rappresentato come una stringa definita su un alfabeto finito in cui ciascun simbolo rappresenta una delle quattro basi azotate. Questa rappresentazione permette di trattare l'informazione genetica come informazione elaborabile computazionalmente, trasformando problemi di natura biologica in problemi algoritmici su stringhe.

Un problema fondamentale consiste nel confrontare due sequenze biologiche allo scopo di individuarne le corrispondenze e le differenze. Sequenze omologhe possono infatti differire a causa di mutazioni. In ambito computazionale tali differenze possono essere studiate mediante un allineamento di sequenze, che mette in corrispondenza i simboli delle due stringhe introducendo, quando necessario, opportuni simboli di gap. Associando a ciascuna possibile corrispondenza un valore attraverso un modello valutativo, il confronto può essere formulato come un problema di ottimizzazione: il Global Alignment Problem, che consiste nel determinare un allineamento avente score massimo.

Una soluzione classica a questo problema è fornita dall'algoritmo di Needleman–Wunsch, basato sulla programmazione dinamica. Tale approccio consente di ricostruire un allineamento ottimo, ma nella sua formulazione standard richiede uno spazio proporzionale al prodotto delle lunghezze delle due sequenze. Per input di grandi dimensioni, il consumo di memoria può quindi diventare un limite pratico significativo.

L'obiettivo di questa tesi è analizzare il problema dell'allineamento globale con particolare attenzione all'utilizzo della memoria, confrontando l'algoritmo di Needleman–Wunsch con l'algoritmo di Hirschberg, che attraverso una strategia divide et impera permette di ricostuire un allineamento ottimo utilizzando spazio lineare. I due algoritmi vengono analizzati teoricamente, implementati in linguaggio C e infine confrontati sperimentalmente su sequenze sintetiche e biologiche reali, valutandone tempo di esecuzione e consumo di memoria.

Indice

Introduzione **iii**

Indice **v**

1 Definizione del Global Alignment Problem e dei prerequisiti essenziali **1**

1.1 Rappresentazione delle sequenze biologiche 1

1.2 Allineamento e modello di scoring 2

1.3 Programmazione dinamica 5

1.4 Longest Common Subsequence 6

1.5 Alignment graph 8

1.6 Global Alignment Problem 10

2 Algoritmo di Needleman–Wunsch **12**

2.1 Ricorrenza, inizializzazione e traceback 12

2.2 Pseudocodice 16

2.3 Analisi della complessità 18

2.4 Implementazione 19

2.4.1 Strutture dati 19

2.4.2 Algoritmo 21

3 Algoritmo di Hirschberg **23**

3.1 Ricorrenza, Middle vertex e Middle edge 25

3.2 Pseudocodice 30

3.3 Analisi della complessità 35

3.4 Implementazione 38

3.4.1 Calcolo degli score in spazio lineare 38
3.4.2 Middle vertex e middle edge 39
3.4.3 Ricostruzione ricorsiva dell'allineamento 42

4 Metodologia sperimentale **43**
4.1 Dataset 43
4.2 Ambiente hardware e software 45
4.3 Misura dei tempi e della memoria 46

5 Risultati e confronto **47**
5.1 Validazione dei risultati 47
5.2 Scalabilità su sequenze sintetiche 48
5.3 Risultati sulle sequenze biologiche reali 52
5.4 Conclusioni, Needleman–Wunsch vs Hirschberg 53

Bibliografia **55**

Capitolo 1

Definizione del Global Alignment Problem e dei prerequisiti essenziali

1.1 Rappresentazione delle sequenze biologiche

Il DNA è costituito da quattro tipi di nucleotidi, caratterizzati dalle seguenti basi azotate:

- Adenina, `A`;
- Citosina, `C`;
- Guanina, `G`;
- Timina, `T`.

I due filamenti della molecola sono complementari e antiparalleli, con appaiamenti

$$A \longleftrightarrow T, \qquad C \longleftrightarrow G$$

Dal punto di vista computazionale, un singolo filamento può essere rappresentato come una stringa sull'alfabeto

$$\Sigma_{\text{DNA}} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$$

Quindi una stringa di lunghezza n ha forma

$$x = x_1 x_2 \cdots x_n, \qquad x_i \in \Sigma_{\text{DNA}} \qquad \forall i \in \{1, \ldots, n\}.$$

Una *sottostringa* è una porzione contigua di una stringa, della forma

$$x_i x_{i+1} \cdots x_j,$$

con $1 \leq i \leq j \leq n$, e n la lunghezza della stringa originale. In una stringa di lunghezza n, una sottostringa di lunghezza k esiste per $k \leq n$.

Un *k-mero* è una sottostringa di lunghezza $k \geq 1$.

Data una stringa, una *sottosequenza* è invece una stringa ottenuta eliminando 0 o più elementi dalla stringa originale mantenendone l'ordine relativo.

Mostriamo un esempio: sia s una stringa che rappresenta una sequenza genomica,

$$s := \texttt{ACCT},$$

allora `AC` può essere sia una sottostringa che una sottosequenza, mentre `AT` è definibile soltanto come sottosequenza.

Due sequenze genomiche omologhe possono differire tra loro a causa di mutazioni. Consideriamo tre eventi possibili associati a tali mutazioni:

$$\begin{aligned} \texttt{AC} &\xrightarrow{\text{sostituzione}} \texttt{AA} \\ \texttt{AC} &\xrightarrow{\text{inserimento}} \texttt{AAC} \\ \texttt{AAC} &\xrightarrow{\text{cancellazione}} \texttt{AC} \end{aligned}$$

Gli ultimi due eventi verranno talvolta trattati come appartenenti a un unico tipo, dal nome *indel* [JP04, pp. 167–172].

Supponiamo, quindi, di avere due sequenze genomiche omologhe che differiscono a causa di mutazioni genetiche; come possiamo quantificare e individuare la somiglianza tra le due sequenze?

1.2 Allineamento e modello di scoring

Stiamo quindi iniziando a definire il problema di nostro interesse.

Dato che un nostro obiettivo è quello di misurare la differenza tra due sequenze genetiche, trattiamo adesso la *distanza di Hamming*, che è lo strumento più semplice per

valutare la somiglianza tra due stringhe. Siano x, y due stringhe della stessa lunghezza, allora la distanza di Hamming è

$$d_H(x, y) := |\{i : x_i \neq y_i\}|,$$

ovvero il numero di posizioni in cui due stringhe differiscono [JP04]. La distanza di Hamming è però applicabile soltanto a stringhe della stessa lunghezza e non modella adeguatamente inserzioni e delezioni, che possono inoltre alterare la corrispondenza tra le posizioni delle due sequenze.

Abbiamo quindi bisogno di una misura più generale. Definiamo allora l'*edit distance*, o anche distanza di Levenshtein, $d(v, w)$, come il numero minimo di operazioni elementari di inserimento, cancellazione e sostituzione necessarie per trasformare una sequenza v in una sequenza w [JP04, pp. 167–172].

L'edit distance fornisce una misura della dissimilarità tra due sequenze. Per rappresentare esplicitamente le corrispondenze tra i loro simboli introduciamo ora il concetto di **allineamento**.

Siano v e w due sequenze. Un allineamento di v e w è una matrice

$$A \in (\Sigma_{\text{DNA}} \cup \{-\})^{2\times L},$$

con

$$\max\{|v|, |w|\} \leq L \leq |v| + |w|,$$

tale che, eliminando i simboli di gap (' - ') dalla prima riga, si ottiene v, mentre eliminandoli dalla seconda si ottiene w. Non sono ammesse colonne contenenti due gap [CP15, pp. 226–228].

Ogni colonna di un allineamento corrisponde a uno dei seguenti casi:

- match (corrispondenza);
- mismatch (sostituzione);
- inserimento;
- cancellazione;

Visualizziamo cosa è stato appena detto: siano $v_{aligned}$ e $w_{aligned}$ le sequenze allineate e A il loro allineamento, allora un esempio potrebbe essere:

$$\begin{matrix} v_{\text{aligned}} & : & \texttt{A} & \texttt{C} & \texttt{T} & \texttt{-} & \texttt{G} & \texttt{C} & \texttt{A} \\ w_{\text{aligned}} & : & \texttt{A} & \texttt{-} & \texttt{T} & \texttt{C} & \texttt{G} & \texttt{C} & \texttt{A} \end{matrix} = A$$

Per valutare la similarità tra simboli introduciamo una funzione di **scoring**,

$$\delta : ((\Sigma_{\text{DNA}} \cup \{-\})^2 \setminus \{(-,-)\}) \to \mathbb{R},$$

che, dati due simboli a e b, restituisce il contributo di score associato al loro allineamento.

Una funzione di scoring (anche detta modello) per una sequenza di DNA è generalmente definita attribuendo un premio in caso di match e una penalità in caso contrario

$$\delta(a,b) = \begin{cases} \mu, & a = b, \qquad a,b \in \Sigma_{\text{DNA}} \\ -\eta, & a \neq b, \qquad a,b \in \Sigma_{\text{DNA}} \\ -\sigma, & \text{se esattamente uno tra } a,b \text{ è un gap} \end{cases}$$

con $\mu, \eta, \sigma > 0$. Il modello in cui mismatch e gap hanno la stessa penalità si ottiene come caso particolare ponendo $\eta = \sigma$ [JP04, pp. 177–180].

Grazie alla funzione di scoring possiamo anche definire la **scoring matrix**,

δ	A	C	G	T	-
A	μ	$-\eta$	$-\eta$	$-\eta$	$-\sigma$
C	$-\eta$	μ	$-\eta$	$-\eta$	$-\sigma$
G	$-\eta$	$-\eta$	μ	$-\eta$	$-\sigma$
T	$-\eta$	$-\eta$	$-\eta$	μ	$-\sigma$
-	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	n.a.

Siano x, y le sequenze allineate corrispondenti rispettivamente a due stringhe v e w, quindi $x_i, y_i \in \Sigma_{\text{DNA}} \cup \{-\}$, $\forall i \in \{1, \ldots, L\}$.

Sia la matrice di allineamento

$$A = \begin{pmatrix} x_1 & \cdots & x_L \\ y_1 & \cdots & y_L \end{pmatrix},$$

allora, definiamo lo score complessivo dell'allineamento come

$$\text{score}(A) = \sum_{i=1}^{L} \delta(x_i, y_i).$$

Il *problema dell'allineamento globale* (Global Alignment Problem) consiste nel trovare, date due sequenze, un allineamento di score massimo.

Nel modello precedente ogni gap viene penalizzato indipendentemente. Una successione di gap consecutivi può tuttavia essere interpretata come risultato di un unico evento di inserimento o cancellazione. Per questo motivo vengono spesso utilizzate penalità affini, che distinguono il costo di apertura di un gap da quello della sua estensione [JP04, p. 184]. Tale generalizzazione richiede stati aggiuntivi nella programmazione dinamica; nel seguito ci concentreremo tuttavia sul modello a penalità lineare utilizzato dagli algoritmi oggetto del confronto.

1.3 Programmazione dinamica

La *programmazione dinamica* è una tecnica algoritmica adatta a problemi caratterizzati da sottoproblemi sovrapposti e da una proprietà di sottostruttura ottima. Le soluzioni dei sottoproblemi vengono memorizzate, evitando di calcolarli più volte [CLRS22, Sec. 14.3].

Nel problema di allineamento che considereremo, i sottoproblemi possono essere indicizzati mediante due indici i e j. In particolare, una ricorrenza avrà tipicamente la forma

$$P(i,j) = F\big(P(i-1,j), P(i,j-1), P(i-1,j-1)\big).$$

Siano $|v| = n$ e $|w| = m$. Indicheremo con

$$S_{i,j}$$

il valore ottimo relativo ai prefissi

$$v_1 \cdots v_i \qquad \text{e} \qquad w_1 \cdots w_j,$$

con $0 \leq i \leq n$ e $0 \leq j \leq m$.

Nel caso dell'allineamento di sequenze, lo stato $S_{i,j}$ dipenderà dai tre stati

$$S_{i-1,j}, \qquad S_{i,j-1}, \qquad S_{i-1,j-1},$$

che corrisponderanno rispettivamente a una cancellazione, un inserimento e a un match o mismatch.

Memorizzando esplicitamente tutti gli stati è necessaria una matrice di $(n+1)\times(m+1)$ elementi, e quindi uno spazio $O(nm)$.

Una volta calcolato il valore ottimo, una soluzione corrispondente può essere ricostruita memorizzando le scelte effettuate e percorrendole a ritroso mediante *traceback*.

1.4 Longest Common Subsequence

Una *sottosequenza comune* a due stringhe v e w è una stringa che è una sottosequenza sia di v sia di w.

Quindi, siano

$$v = v_1 v_2 \cdots v_n, \qquad w = w_1 w_2 \cdots w_m.$$

Una stringa

$$z = z_1 z_2 \cdots z_k$$

è una sottosequenza comune a v e w se esistono indici

$$1 \leq i_1 < i_2 < \cdots < i_k \leq n$$

e

$$1 \leq j_1 < j_2 < \cdots < j_k \leq m$$

tali che

$$z_t = v_{i_t} = w_{j_t}, \qquad \forall t \in \{1, \ldots, k\}$$

Facciamo un esempio per chiarire:

Siano

$$v := \texttt{ACCT}, \qquad w := \texttt{ACT}.$$

Allora, la stringa

$$z = \texttt{ACT}$$

è una sottosequenza comune.

Tra tutte le sottosequenze comuni a v e w, siamo interessati ad una di lunghezza massima, detta **Longest Common Subsequence** (LCS) [JP04, pp. 172–177].

Indichiamo con $s(v, w)$ la lunghezza di una LCS tra v e w:

$$s(v, w) = \max\{|z| : z \text{ è una sottosequenza comune di } v \text{ e } w\}$$

Considerando una variante dell'edit distance in cui sono ammesse soltanto inserzioni e delezioni, trasformare v in w utilizzando una sottosequenza comune di lunghezza k richiede

$$(n-k)+(m-k)$$

operazioni. Per una LCS, con $k = s(v,w)$, si ottiene quindi tale distanza come

$$d_{\text{indel}}(v,w) = n + m - 2s(v,w)$$

Riprendiamo l'esempio precedente utilizzando le stesse sequenze v e w. La stringa

$$z = \texttt{ACT}$$

è una sottosequenza di entrambe le sequenze ma è inoltre una LCS, poiché $|w| = 3$ e non può quindi esistere una sottosequenza comune più lunga.

Pertanto,

$$s(v,w) = 3$$

e

$$d_{\text{indel}}(v,w) = 4 + 3 - 2 \times 3 = 1.$$

Il problema della LCS può essere ricondotto alla ricerca di un cammino di peso massimo da $(0,0)$ a (n,m) su un digrafo aciclico definito in modo appropriato come segue [JP04, pp. 172–177].

A ogni coppia di prefissi

$$v_1 \cdots v_i, \qquad w_1 \cdots w_j$$

associamo un nodo (i,j).

Ogni nodo (i,j) può avere tre tipi di archi entranti:

- $(i-1,j) \to (i,j)$ (arco 'verticale', corrispondente all'omissione di un simbolo di v);
- $(i,j-1) \to (i,j)$ (arco 'orizzontale', corrispondente all'omissione di un simbolo di w);
- $(i-1,j-1) \to (i,j)$, soltanto quando $v_i = w_j$ (arco 'diagonale', corrispondente a un match).

Nel problema della LCS assegniamo un peso a ogni arco, $c(e)$, dove $e \in E$, con E insieme degli archi della griglia.

$$c(e) = \begin{cases} 1, & \text{arco diagonale corrispondente a un match,} \\ 0, & \text{arco verticale o orizzontale} \end{cases}$$

Il problema della LCS può quindi essere ricondotto alla ricerca di un cammino di peso massimo da $(0,0)$ a (n,m) nel DAG associato alla griglia descritta sopra.

Sia $S_{i,j}$ la lunghezza di una LCS tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$. Siano

$$S_{i,0} = 0, \qquad S_{0,j} = 0.$$

Allora vale la ricorrenza:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} \\ S_{i,j-1} \\ S_{i-1,j-1} + 1 \quad \text{se } v_i = w_j \end{cases}$$

I primi due termini corrispondono rispettivamente a non utilizzare v_i oppure w_j nella sottosequenza comune. Se invece $v_i = w_j$, è possibile utilizzare entrambi i simboli aggiungendo 1 alla soluzione relativa ai prefissi precedenti. La struttura della ricorrenza riprende quindi la dipendenza dai tre sottoproblemi introdotti nella sezione precedente.

Poiché gli stati sono $(n+1) \times (m+1)$, memorizzare l'intera matrice richiede spazio

$$O(nm).$$

Poiché ogni stato viene calcolato utilizzando un numero costante di operazioni, anche il tempo di calcolo è

$$O(nm).$$

Se interessa soltanto la lunghezza della LCS, è sufficiente mantenere in memoria le righe necessarie al calcolo, riducendo così lo spazio a $O(\min(n,m))$ [JP04, p. 230].

1.5 Alignment graph

La rappresentazione tramite grafo introdotta per la LCS può essere generalizzata al problema dell'allineamento di sequenze [JP04, pp. 167–177].

Siano v e w due sequenze, con $|v| = n$ e $|w| = m$. Definiamo l'**alignment graph** come il grafo diretto aciclico i cui nodi sono le coppie

$$(i, j), \qquad 0 \leq i \leq n, \quad 0 \leq j \leq m,$$

dove il nodo (i, j) rappresenta l'allineamento dei prefissi

$$v_1 \cdots v_i, \qquad w_1 \cdots w_j,$$

e i cui archi sono definiti come segue:

Direzione	Arco	Operazione nell'allineamento
verticale	$(i-1, j) \to (i, j)$	cancellazione / v_i contro gap
orizzontale	$(i, j-1) \to (i, j)$	inserimento / gap contro w_j
diagonale	$(i-1, j-1) \to (i, j)$	match o mismatch

Lo stato iniziale è $(0, 0)$ e quello finale è (n, m).

Ogni allineamento tra v e w corrisponde a un cammino nell'alignment graph da $(0, 0)$ a (n, m) e, viceversa, ogni cammino tra questi nodi rappresenta un possibile allineamento.

A questo punto è importante mostrare un esempio per visualizzare l'alignment graph. Siano $v := \texttt{ACCT}, w := \texttt{ACT}$ due sequenze. Sia A il loro allineamento. Allora possiamo elencare gli archi che compongono un cammino da $(0, 0)$ a $(4, 3)$ nell'alignment graph associato ad A:

$$(0, 0) \to (1, 1) \to (2, 2) \to (3, 2) \to (4, 3).$$

Gli archi corrispondono a:

arco	colonna dell'allineamento
$(0, 0) \to (1, 1)$	(A, A)
$(1, 1) \to (2, 2)$	(C, C)
$(2, 2) \to (3, 2)$	$(C, -)$
$(3, 2) \to (4, 3)$	(T, T)

producendo quindi:

$$A = \begin{matrix} \texttt{A} & \texttt{C} & \texttt{C} & \texttt{T} \\ \texttt{A} & \texttt{C} & \texttt{-} & \texttt{T} \end{matrix}$$

Il peso di ogni arco è definito come segue:

$$w\big((i-1,j)\to(i,j)\big)=\delta(v_i,-),$$
$$w\big((i,j-1)\to(i,j)\big)=\delta(-,w_j),$$
$$w\big((i-1,j-1)\to(i,j)\big)=\delta(v_i,w_j),$$

con δ la funzione di scoring.

Il peso di un cammino coincide quindi con lo score dell'allineamento corrispondente.

1.6 Global Alignment Problem

Definiamo ora formalmente il problema dell'allineamento globale, che costituisce il problema di riferimento per gli algoritmi analizzati nel seguito.

Siano v e w due sequenze e sia δ la funzione di scoring definita precedentemente.

Dato un allineamento

$$A=\begin{pmatrix} x_1 & x_2 & \cdots & x_r \\ y_1 & y_2 & \cdots & y_r \end{pmatrix}$$

costituito da r colonne, definiamo il suo score complessivo come

$$\text{score}(A)=\sum_{k=1}^{r}\delta(x_k,y_k)$$

Sia $\mathcal{A}(v,w)$ l'insieme di tutti i possibili allineamenti tra v e w. Il **Global Alignment Problem** consiste nel determinare un allineamento di score massimo, [JP04, pp. 177–180], ovvero

$$\text{OPT}(v,w)=\max_{A\in\mathcal{A}(v,w)}\text{score}(A)$$

Nel modello di scoring lineare considerato di seguito,

$$\text{match}=+1,\qquad \text{mismatch}=-\eta,\qquad \text{indel}=-\sigma,$$

con $\eta,\sigma>0$.

Lo score di un allineamento può quindi essere espresso come

$$\text{score}(A)=\#\text{match}-\eta\#\text{mismatch}-\sigma\#\text{gap},$$

dove #gap indica il numero di colonne dell'allineamento contenenti un gap dovuto a una trasformazione indel.

Per quanto osservato nella sezione precedente, ogni allineamento corrisponde a un cammino da $(0, 0)$ a (n, m) nell'alignment graph e il peso del cammino coincide con lo score dell'allineamento. Il Global Alignment Problem può quindi essere interpretato come la ricerca di un cammino di peso massimo tra questi due nodi.

Da questa formulazione emergono tre obiettivi computazionali che guideranno l'analisi degli algoritmi successivi:

- calcolare il valore ottimo $\mathrm{OPT}(v, w)$;
- ricostruire un allineamento che realizzi tale valore;
- ricostruire un allineamento ottimo riducendo l'utilizzo di memoria.

Capitolo 2

Algoritmo di Needleman–Wunsch

In questo capitolo analizzeremo l'algoritmo di Needleman–Wunsch [NW70], che è un algoritmo di programmazione dinamica per il problema dell'allineamento globale di sequenze.

2.1 Ricorrenza, inizializzazione e traceback

Sia $S_{i,j}$ il miglior score globale tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$, con $0 \leq i \leq n, 0 \leq j \leq m$. Quindi, come visto nel precedente capitolo, possiamo affermare che $S_{i,j}$ coincide con il peso di un cammino massimo da $(0,0)$ a (i,j).

Consideriamo l'ultima colonna di un allineamento ottimo, abbiamo tre possibilità:

- **Cancellazione**

  L'ultima colonna è

  $$\begin{pmatrix} v_i \\ - \end{pmatrix}$$

  quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_j$.

  Lo score in questo caso è

  $$S_{i-1,j} + \delta(v_i, -)$$

- **Inserimento**

  L'ultima colonna è

  $$\begin{pmatrix} - \\ w_j \end{pmatrix}$$

  quindi prima di essa è stato allineato $v_1 \cdots v_i$ con $w_1 \cdots w_{j-1}$.

  Lo score in questo caso è

  $$S_{i,j-1} + \delta(-, w_j)$$

- **Match o mismatch**

  L'ultima colonna è

  $$\begin{pmatrix} v_i \\ w_j \end{pmatrix}$$

  quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_{j-1}$.

  Lo score in questo caso è

  $$S_{i-1,j-1} + \delta(v_i, w_j)$$

Per gli stati interni dell'allineamento, con $1 \leq i \leq n$ e $1 \leq j \leq m$, possiamo formulare la ricorrenza a partire dai tre possibili predecessori. Prima però è necessario inizializzare gli stati sul bordo della matrice, che corrispondono ai casi base del problema dell'allineamento globale:

$$\begin{aligned} S_{0,0} &= 0; \\ S_{i,0} &= S_{i-1,0} + \delta(v_i, -) = -i\sigma; \\ S_{0,j} &= S_{0,j-1} + \delta(-, w_j) = -j\sigma. \end{aligned}$$

A differenza della LCS, gli stati sul bordo non hanno valore nullo: nel global alignment l'intero prefisso deve essere allineato. Allineare $v_1 \cdots v_i$ con la stringa vuota richiede quindi i gap, mentre allineare la stringa vuota con $w_1 \cdots w_j$ ne richiede j.

Possiamo, quindi, formalizzare la *ricorrenza*:

$$\boxed{S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -), \\ S_{i,j-1} + \delta(-, w_j), \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}}$$

con $1 \leq i \leq n$ e $1 \leq j \leq m$.

Questa relazione costituisce la ricorrenza fondamentale dell'algoritmo di Needleman–Wunsch [CP15, pp. 255–256].

Il calcolo di $S_{n,m}$ soddisfa il primo degli obiettivi formulati nel capitolo precedente, fornendo

$$S_{n,m} = \mathrm{OPT}(v, w).$$

Per ricostruire anche un allineamento che realizzi tale score è necessario mantenere l'informazione relativa alle scelte effettuate durante il calcolo [CP15, pp. 251–252]. Quindi, durante il calcolo di $S_{i,j}$, possiamo memorizzare quale predecessore ha prodotto il massimo all'interno di una matrice B:

$$B_{i,j} \in \{\uparrow, \leftarrow, \nwarrow\}.$$

Sul bordo della matrice il predecessore è univocamente determinato:

$$B_{i,0} = \uparrow, \qquad i = 1, \ldots, n,$$
$$B_{0,j} = \leftarrow, \quad j = 1, \ldots, m.$$

A ciascuna direzione memorizzata in $B_{i,j}$ corrisponde una colonna dell'allineamento secondo la seguente associazione:

$B_{i,j}$	colonna dell'allineamento
$\uparrow$	$(v_i, -)$
$\leftarrow$	$(-, w_j)$
$\nwarrow$	(v_i, w_j)

Così facendo, partendo da (n, m) possiamo ricostruire l'allineamento percorrendo i predecessori fino a $(0, 0)$. Notiamo però che tramite questo processo di *traceback* le colonne vengono generate in ordine contrario, quindi al termine vanno invertite.

In caso di parità tra più direzioni esistono più cammini equivalenti, e quindi potenzialmente più allineamenti ottimi. Se l'obiettivo è ricostruire un singolo allineamento ottimo, è sufficiente adottare una regola deterministica di tie-breaking; per rappresentare tutti gli allineamenti ottimi sarebbe invece necessario conservare tutti i predecessori che realizzano il massimo.

Mostriamo un esempio. Siano $v := \texttt{ACCT}$ e $w := \texttt{ACT}$ due sequenze. Sia $\delta(a, b)$ la funzione di scoring definita in modo tale che restituisca 1 in caso di match e -1 altrimenti. Allora la matrice degli score è:

	-	A	C	T
-	0	-1	-2	-3
A	-1	1	0	-1
C	-2	0	2	1
C	-3	-1	1	1
T	-4	-2	0	2

Quindi,

$$S_{4,3} = 2$$

Il traceback sarà:

$$(4, 3) \to (3, 2) \to (2, 2) \to (1, 1) \to (0, 0),$$

ricostruendo

A C C T
A C - T

2.2 Pseudocodice

Possiamo adesso fornire lo pseudocodice per l'algoritmo di Needleman–Wunsch.

Listing 2.1: Global Alignment

```
NeedlemanWunsch(v,w,delta)
    n <- |v|
    m <- |w|

    // inizializzazione
    S[0,0] <- 0
    for i <- 1,...,n
        S[i,0] <- S[i-1,0] + delta(v[i],'-')
        B[i,0] <- up
    for j <- 1,...,m
        S[0,j] <- S[0,j-1] + delta('-',w[j])
        B[0,j] <- left

    // ricorrenza
    for i <- 1,...,n
        for j <- 1,...,m
            up       <- S[i-1,j] + delta(v[i],'-')
            left     <- S[i,j-1] + delta('-',w[j])
            diagonal <- S[i-1,j-1] + delta(v[i],w[j])
            S[i,j] <- max(up, left, diagonal)
            B[i,j] <- predecessore che ha prodotto il massimo

    // ricostruzione dell'allineamento
    alignment <- Backtrack(B,v,w,n,m)

    return (S[n,m] , alignment)
```

Adottiamo una politica deterministica di tie-breaking che assegna priorità, nell'ordine, allo spostamento diagonale, verticale e orizzontale. Il seguente è un possibile pseudocodice del traceback.

Listing 2.2: Ricostruzione dell'allineamento tramite traceback

```
Backtrack(B,v,w,i,j)
    alignedV <- empty
    alignedW <- empty

    while i > 0 or j > 0
        if B[i,j] = diagonal
            append v[i] to alignedV
            append w[j] to alignedW
            i <- i - 1
            j <- j - 1

        else if B[i,j] = up
            append v[i] to alignedV
            append '-' to alignedW
            i <- i - 1

        else
            append '-' to alignedV
            append w[j] to alignedW
            j <- j - 1

    reverse alignedV
    reverse alignedW

    return (alignedV,alignedW)
```

Quindi, la prima procedura costruisce la matrice degli score e quella dei predecessori; la seconda utilizza quest'ultima per ricostruire un allineamento ottimo a partire dalla cella (n, m).

2.3 Analisi della complessità

Abbiamo allora definito l'algoritmo standard di Needleman–Wunsch. Possiamo adesso analizzarne la complessità, centro del nostro confronto.

Sia $T(n, m)$ la complessità di esecuzione, temporale, dell'algoritmo. Sia $S(n, m)$ la complessità spaziale dell'algoritmo.

Allora,

$$T(n, m) = \Theta(nm),$$

poiché la matrice contiene $(n + 1) \times (m + 1)$ stati e ciascuno viene calcolato mediante un numero costante di operazioni.

Memorizzando l'intera matrice avremo

$$S(n, m) = \Theta(nm).$$

Pertanto, per sequenze di lunghezza dello stesso ordine, tempo e spazio crescono quadraticamente rispetto alla lunghezza.

La crescita quadratica dello spazio può quindi diventare un limite per sequenze di grandi dimensioni, poiché memorizzare l'intera matrice richiede una quantità di memoria proporzionale al prodotto delle lunghezze delle due sequenze.

Inoltre, non abbiamo ancora considerato il costo del traceback. Un cammino da (n, m) a $(0, 0)$ contiene al più $n+m$ archi, quindi il traceback percorre al massimo $n+m$ celle, richiedendo tempo

$$T_{\text{traceback}}(n, m) = O(n + m).$$

Inoltre, l'allineamento prodotto può contenere al massimo $n + m$ colonne, e richiede quindi

$$O(n + m)$$

spazio per la memorizzazione dell'output.

Pertanto, il costo temporale complessivo dell'algoritmo rimane

$$T(n, m) = \Theta(nm) + O(n + m) = \Theta(nm),$$

mentre la complessità spaziale complessiva della versione standard è

$$S(n, m) = \Theta(nm) + O(n + m) = \Theta(nm).$$

Sappiamo che, per calcolare

$$S_{i,j}$$

abbiamo bisogno di

$$S_{i-1,j}, \qquad S_{i,j-1}, \qquad S_{i-1,j-1}.$$

Allora, possiamo notare come il calcolo della colonna j richieda soltanto la colonna $j-1$ e i valori già calcolati della colonna j. Quindi, se riduciamo il problema soltanto al calcolo del punteggio ottimo, potremmo diminuire lo spazio necessario, andando ad usare soltanto due vettori di $n+1$ o $m+1$ elementi, scegliendo opportunamente la sequenza più corta, ottenendo quindi uno spazio lineare [JP04, pp. 230–231].

$$S_{\text{score}}(n,m) = \Theta(\min\{n,m\}).$$

Notiamo che la complessità nel tempo non cambia.

Tale ottimizzazione consente quindi di soddisfare il primo obiettivo computazionale, ossia il calcolo dello score ottimo, utilizzando spazio lineare. Tuttavia, eliminando le celle già utilizzate vengono perse anche le informazioni necessarie per effettuare il traceback e ricostruire un allineamento ottimo. Il problema diventa quindi quello di ottenere anche la soluzione, e non soltanto il suo valore, mantenendo una complessità spaziale lineare.

2.4 Implementazione

L'algoritmo di Needleman–Wunsch è stato implementato in linguaggio C mantenendo una corrispondenza diretta tra gli oggetti introdotti nella formulazione teorica e le strutture dati utilizzate nel programma. In questa sezione vengono descritte le principali scelte implementative rilevanti ai fini del confronto sperimentale.

2.4.1 Strutture dati

Per rappresentare i simboli di una sequenza di DNA è stato definito il tipo `Base`, che fornisce una codifica numerica degli elementi di

$$\Sigma_{\text{DNA}} \cup \{-\},$$

includendo quindi le quattro basi azotate e un valore dedicato al gap.

Un filamento di DNA è rappresentato dal tipo `Strand`, contenente un array di tipo `Base` e la lunghezza di tale array.

La funzione associata all'algoritmo di Needleman–Wunsch restituisce un oggetto di tipo

`Alignment`,

formato da due sequenze, `Strand`, e da un intero, `score`, che corrisponde a $S_{n,m}$.

Poiché nella formulazione teorica le sequenze sono indicizzate a partire da 1, mentre gli array C partono da 0, al simbolo v_i corrisponde nel codice `v->bases[i-1]`, e analogamente w_j corrisponde a `w->bases[j-1]`.

Nel programma è stato utilizzato il seguente modello di scoring di default:

$$\texttt{match} = 1, \qquad \texttt{mismatch} = -1, \qquad \texttt{indel} = -1.$$

La funzione

`scoring(a,b,model)`

implementa la funzione di scoring teorica $\delta(a, b)$; il parametro `model` permette di specificare un diverso modello di scoring.

L'implementazione alloca due matrici, `score` e `pred`, entrambe di dimensione $(n + 1) \times (m + 1)$. La prima memorizza gli score $S_{i,j}$ associati agli stati della programmazione dinamica, mentre la seconda memorizza le informazioni relative ai predecessori.

$$\texttt{score} \leftrightarrow S, \qquad \texttt{pred} \leftrightarrow B.$$

Entrambe le matrici sono allocate dinamicamente come blocchi contigui di memoria. Questa scelta consente di mantenere la rappresentazione bidimensionale `score[i][j]`, limitando l'overhead dovuto a molteplici allocazioni e garantendo una disposizione regolare degli elementi in memoria.

La matrice `pred` utilizza il tipo `Direction`; le possibili direzioni sono rappresentate mediante flag distinti e combinate attraverso una maschera di bit. In questo modo una singola cella può memorizzare contemporaneamente più predecessori ottimi.

2.4.2 Algoritmo

Dopo l'inizializzazione della prima riga e della prima colonna delle matrici, gli stati interni vengono calcolati secondo la ricorrenza descritta nella Sezione 2.1. Il seguente frammento mostra il riempimento delle matrici `score` e `pred`.

```
for(size_t i=1; i<=n; i++) {
    for(size_t j=1; j<=m; j++) {
        int up = score[i-1][j] + scoring(v->bases[i-1],BASE_GAP,model);
        int left = score[i][j-1] + scoring(BASE_GAP,w->bases[j-1],model);
        int diagonal = score[i-1][j-1] + scoring(v->bases[i-1],w->bases[j-1],
model);
        score[i][j] = max(up,left,diagonal);

        pred[i][j] = NONE;
        if(score[i][j] == up) pred[i][j]|=UP;
        if(score[i][j] == left) pred[i][j]|=LEFT;
        if(score[i][j] == diagonal) pred[i][j]|=DIAGONAL;
    }
}
```

A differenza dello pseudocodice della Sezione 2.2, la scelta del predecessore non viene effettuata durante il riempimento della matrice. In caso di parità, tutte le direzioni che realizzano il massimo vengono memorizzate in `pred[i][j]` mediante una bitmask. L'operatore OR bit a bit consente infatti di aggiungere un flag senza eliminare quelli già presenti.

La selezione di un singolo predecessore avviene soltanto durante il traceback, secondo la politica deterministica

$$\texttt{DIAGONAL} \succ \texttt{UP} \succ \texttt{LEFT}.$$

Poiché il traceback procede dalla cella (n, m) verso $(0, 0)$, le colonne dell'allineamento vengono generate in ordine inverso. Al termine della ricostruzione le due sequenze vengono quindi invertite mediante

```
reverse(aligned_v,length);
reverse(aligned_w,length);
```

La funzione `needleman_wunsch` restituisce un oggetto di tipo `Alignment`, dove

$$\texttt{alignment.score = score[n][m]} = \mathrm{OPT}(v, w).$$

Le matrici `score` e `pred` costituiscono strutture ausiliarie necessarie soltanto durante l'esecuzione dell'algoritmo.

Dopo il traceback, una volta trasferiti nella struttura `Alignment` lo score finale e le sequenze allineate, la memoria associata alle due matrici può essere liberata.

```
free(score);
free(pred);
```

L'implementazione riflette quindi direttamente la complessità spaziale analizzata nella sezione precedente: sia `score` sia `pred` richiedono un numero di celle proporzionale a $(n+1) \times (m+1)$. Sarà proprio l'eliminazione della necessità di conservare tali matrici complete a motivare l'algoritmo di Hirschberg.

Capitolo 3

Algoritmo di Hirschberg

Nel capitolo precedente abbiamo osservato che, se siamo interessati soltanto al calcolo dello score ottimo del problema dell'allineamento globale, non è necessario memorizzare l'intera matrice della programmazione dinamica. Mantenendo solamente le colonne necessarie al calcolo dello stato successivo, lo spazio ausiliario può infatti essere ridotto a

$$\Theta(\min\{n, m\}).$$

Questa ottimizzazione, tuttavia, non permette direttamente di ricostruire un allineamento ottimo, poiché le informazioni necessarie al traceback vengono progressivamente eliminate prima che possano essere utilizzate.

L'algoritmo proposto da Hirschberg [Hir75] nasce nel contesto del problema della Longest Common Subsequence e utilizza una strategia divide et impera per consentire la ricostruzione della soluzione in spazio lineare. Nel seguito consideriamo la corrispondente formulazione per il problema dell'allineamento globale di sequenze [JP04, Sec. 7.2].

L'obiettivo è mantenere una complessità spaziale lineare, ricostruendo comunque un allineamento associato allo score massimo.

Per ottenere questo risultato, Hirschberg combina il calcolo degli score in spazio lineare con una strategia *divide et impera* applicata all'alignment graph. Consideriamo infatti un cammino ottimo da $(0, 0)$ (sorgente) a (n, m) (pozzo). Tale cammino deve attraversare la colonna centrale del grafo,

$$j_{\text{mid}} = \left\lfloor \frac{m}{2} \right\rfloor .$$

Se riusciamo quindi a individuare un vertice della colonna j_{mid} appartenente ad almeno un cammino ottimo, possiamo suddividere il problema in due sottoproblemi: la ricerca di un cammino ottimo dalla sorgente a tale vertice e la ricerca di un cammino ottimo dal vertice al pozzo [JP04, pp. 232–234].

Applicando ricorsivamente questa procedura è possibile ricostruire il cammino ottimo senza memorizzare l'intera matrice dei predecessori.

Nella formulazione utilizzata nel seguito, l'algoritmo individua non soltanto un vertice centrale, detto *middle vertex*, ma anche un arco ottimo uscente da esso, detto *middle edge*. Tale arco permette di determinare una colonna dell'allineamento e di separare precisamente i due sottoproblemi ricorsivi.

È importante sottolineare che il middle edge è una scelta formulativa dell'algoritmo. La formulazione originale di Hirschberg e la presentazione di Jones e Pevzner [Hir75, JP04] effettuano infatti la decomposizione utilizzando il solo middle vertex: una volta individuato un vertice centrale appartenente a un cammino ottimo, si risolvono ricorsivamente i sottoproblemi posti prima e dopo tale vertice.

La formulazione basata sul middle edge, adottata nel seguito costituisce una variante della stessa strategia. Selezionando anche un arco ottimo uscente dal middle vertex, una colonna dell'allineamento viene determinata immediatamente e i due sottoproblemi possono essere posti strettamente prima e dopo tale arco. La loro area complessiva risulta pertanto leggermente inferiore rispetto alla decomposizione basata sul solo vertice [CP15, pp. 275–276]. Tale vantaggio riguarda tuttavia soltanto i fattori costanti: entrambe le formulazioni mantengono tempo $\Theta(nm)$ e spazio lineare. Non ne segue quindi, in generale, una maggiore efficienza pratica, che dipende dal costo necessario per individuare il middle edge.

3.1 Ricorrenza, Middle vertex e Middle edge

La ricorrenza utilizzata nell'algoritmo di Hirschberg è la stessa introdotta per Needleman–Wunsch. Sia $S_{i,j}$ lo score ottimo dell'allineamento tra i prefissi $v_1 \cdots v_i$ e $w_1 \cdots w_j$. Per $i, j > 0$ vale quindi

$$S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -), \\ S_{i,j-1} + \delta(-, w_j), \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

Anche il modello di scoring δ rimane invariato rispetto all'implementazione di Needleman–Wunsch. In questo modo i due algoritmi risolvono lo stesso problema di ottimizzazione e possono successivamente essere confrontati utilizzando le medesime condizioni sperimentali.

La prima osservazione su cui si basa Hirschberg riguarda il calcolo degli score. Per determinare una cella $S_{i,j}$ sono necessari solamente i valori

$$S_{i-1,j}, \qquad S_{i,j-1}, \qquad S_{i-1,j-1}.$$

Di conseguenza, durante il calcolo della colonna j è sufficiente conservare la colonna precedente e i valori già calcolati della colonna corrente.

Senza perdita di generalità, supponiamo $n \leq m$, disponendo quindi la sequenza più corta lungo le righe dell'alignment graph. Gli score possono essere memorizzati in due vettori

$$\texttt{previous}, \qquad \texttt{current},$$

entrambi contenenti $n + 1$ elementi.

Una volta terminato il calcolo di una colonna, i ruoli dei due vettori vengono scambiati: la colonna appena calcolata diventa la colonna precedente utilizzata per determinare quella successiva. Non è quindi necessario conservare le colonne già elaborate.

Consideriamo adesso la colonna centrale

$$j_{\text{mid}} = \left\lfloor \frac{m}{2} \right\rfloor.$$

Ogni cammino dalla sorgente $(0, 0)$ al pozzo (n, m) attraversa almeno un vertice della colonna j_{mid}. Vogliamo determinare quale di questi vertici appartiene ad almeno un cammino ottimo.

Per ogni $i \in \{0, \ldots, n\}$ definiamo

$$\text{prefix}(i)$$

come lo score del miglior cammino dalla sorgente $(0, 0)$ al vertice (i, j_{mid}).

In maniera analoga, definiamo

$$\text{suffix}(i)$$

come lo score del miglior cammino dal vertice (i, j_{mid}) al pozzo (n, m).

Definiamo quindi

$$\text{length}(i) = \text{prefix}(i) + \text{suffix}(i).$$

La quantità $\text{length}(i)$ rappresenta il massimo score ottenibile tra tutti i cammini dalla sorgente al pozzo vincolati a passare attraverso (i, j_{mid}).

Poiché ogni cammino dalla sorgente al pozzo deve attraversare la colonna j_{mid}, lo score ottimo globale può essere espresso come

$$\text{OPT} = \max_{0 \le i \le n} \{\text{prefix}(i) + \text{suffix}(i)\}.$$

Allora, scegliendo

$$i_{\text{mid}} \in \arg\max_{0 \le i \le n} \text{length}(i),$$

il vertice $(i_{\text{mid}}, j_{\text{mid}})$ appartiene ad almeno un cammino ottimo, e viene chiamato *middle vertex*.

La decomposizione è possibile grazie alla sottostruttura ottima del problema: se il prefisso o il suffisso di un cammino ottimo, che attraversa tale vertice, non fosse a sua volta ottimo per il corrispondente sottoproblema, sarebbe possibile sostituirlo con un cammino di score maggiore e ottenere uno score complessivo superiore, in contraddizione con l'ottimalità del cammino originario.

I valori $\text{prefix}(i)$ possono essere calcolati mediante la programmazione dinamica in spazio lineare descritta precedentemente, arrestando il calcolo alla colonna j_{mid}.

Per ottenere $\text{suffix}(i)$ possiamo applicare lo stesso procedimento nel verso opposto. Concettualmente, ciò equivale a considerare l'alignment graph con gli archi invertiti e a calcolare il miglior cammino dal pozzo verso la colonna centrale.

Quindi, è sufficiente eseguire la stessa ricorrenza percorrendo le sequenze in ordine inverso, come se avessimo invertito la direzione degli archi.

Una volta determinato il middle vertex rimane da individuare quale arco uscente da esso appartenga a un cammino ottimo [CP15, pp. 275–276].

Dal middle vertex possono partire al più tre archi, corrispondenti alle tre operazioni dell'allineamento.

Sia $R(i, j)$ lo score del miglior cammino dal vertice (i, j) al pozzo. Siano $e_{\downarrow}, e_{\rightarrow}, e_{\searrow}$ i tre archi (se esistono) che hanno come coda il vertice (i, j). Allora, definiamo il costo degli archi appena definiti

$$\begin{aligned} c(e_{\downarrow}) &= \delta(v_{i+1}, -) + R(i+1, j), \\ c(e_{\rightarrow}) &= \delta(-, w_{j+1}) + R(i, j+1), \\ c(e_{\searrow}) &= \delta(v_{i+1}, w_{j+1}) + R(i+1, j+1), \end{aligned}$$

Il middle edge è l'arco che massimizza il costo; in caso di pareggio bisognerà introdurre una politica deterministica per la scelta dell'arco.

Il termine relativo al cammino dalla sorgente al middle vertex non compare nel confronto, poiché è lo stesso per tutti gli archi uscenti da tale vertice. È quindi sufficiente confrontare il peso dell'arco con lo score ottimo del cammino che parte dal suo estremo finale.

Una volta individuato il middle edge, il problema viene suddiviso nei due sottoproblemi posti rispettivamente prima e dopo tale arco.

La ricostruzione dell'allineamento assume quindi la forma

$$A = A_{\text{left}} \circ e_{\text{mid}} \circ A_{\text{right}},$$

dove A_{left} e A_{right} sono gli allineamenti ottenuti ricorsivamente nei due sottoproblemi, mentre e_{mid} determina una singola colonna dell'allineamento complessivo.

In questo modo Hirschberg non necessita di una matrice globale dei predecessori; le colonne dell'allineamento vengono prodotte progressivamente durante la ricorsione.

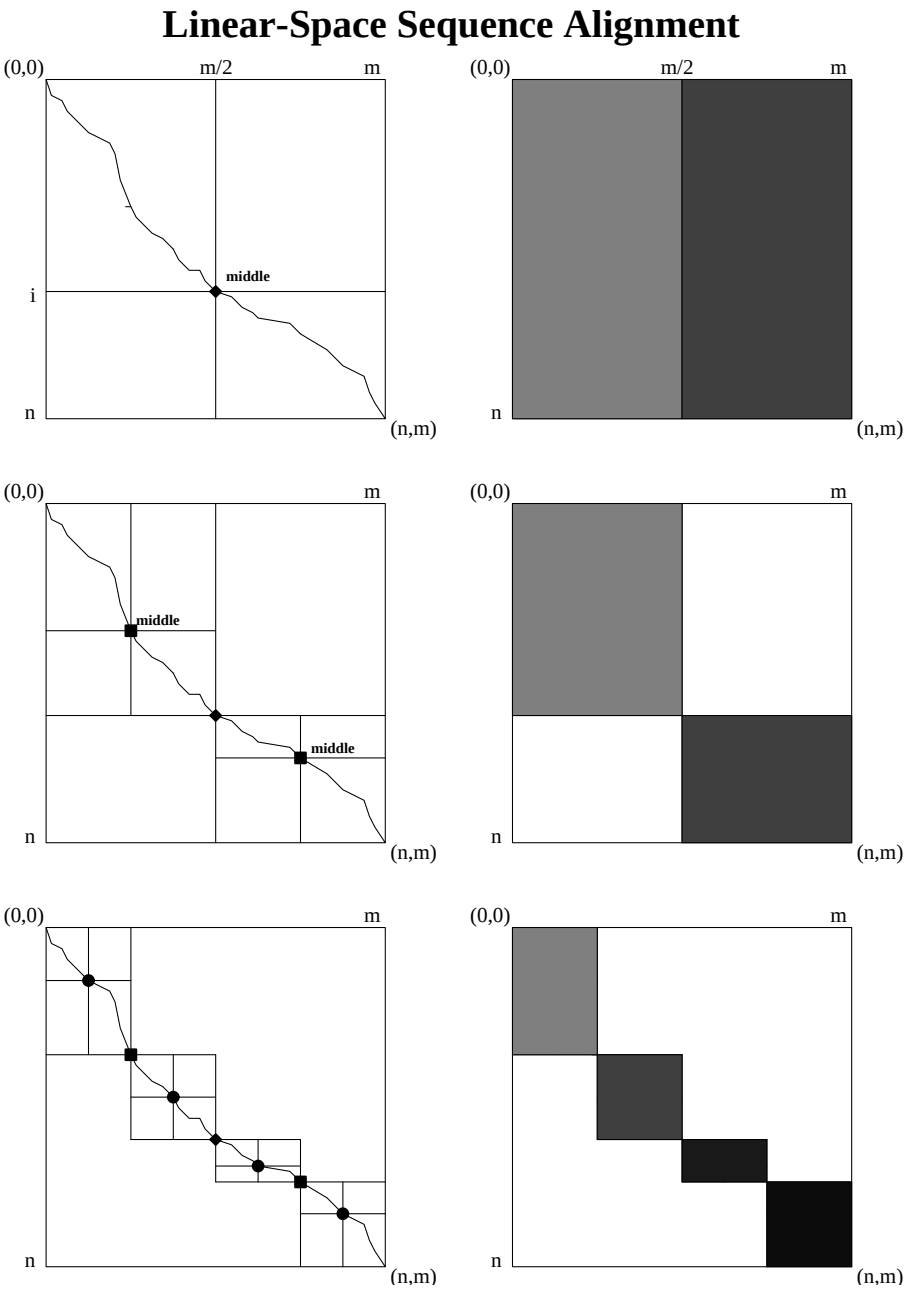


Figura 3.1: Suddivisione ricorsiva dello spazio di allineamento nell'algoritmo in spazio lineare. Tratto da Jones e Pevzner [JP04, p. 235].

Analizziamo un esempio per verificare ciò che abbiamo appena detto. Consideriamo le solite sequenze $v := \texttt{ACCT}$ e $w := \texttt{ACT}$. La colonna centrale è:

$$j_{\text{mid}} = \left\lfloor \frac{3}{2} \right\rfloor = 1$$

I calcoli degli score in avanti e in senso inverso restituiscono:

$$\text{prefix} = \begin{bmatrix} -1 \\ 1 \\ 0 \\ -1 \\ -2 \end{bmatrix}, \qquad \text{suffix} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ -2 \end{bmatrix}$$

Allora,

$$\text{length} = \begin{bmatrix} -1 \\ 2 \\ 2 \\ -1 \\ -4 \end{bmatrix}$$

Abbiamo due possibili scelte per il middle vertex, $(1,1)$ e $(2,1)$. Senza perdita di generalità adottiamo

$$(i_{\text{mid}}, j_{\text{mid}}) = (1,1)$$

Calcoliamo adesso il middle edge dal vertice $(1,1)$

$$c(e_{\downarrow}) = \delta(C,-) + R(2,1) = -1 + 2 = 1,$$

$$c(e_{\rightarrow}) = \delta(-,C) + R(1,2) = -1 - 1 = -2,$$

$$c(e_{\searrow}) = \delta(C,C) + R(2,2) = 1 + 0 = 1.$$

Anche questa volta troviamo un pareggio; adottando la politica di tie-breaking della Sezione 2.4.2, l'arco scelto è

$$(1,1) \to (2,2)$$

Infine, possiamo visualizzare il paradigma divide et impera nel nostro esempio:

$$(0,0) \rightsquigarrow (1,1) \quad \underbrace{(1,1) \to (2,2)}_{\text{middle edge}} \quad (2,2) \rightsquigarrow (4,3)$$

3.2 Pseudocodice

Possiamo unire i risultati precedenti nello pseudocodice dell'algoritmo di Hirschberg [CP15, pp. 269–276].

Sia (i, j) il vertice dell'alignment graph che rappresenta l'allineamento dei prefissi $v_1 \cdots v_i$ e $w_1 \cdots w_j$.

Un sottoproblema viene identificato da un rettangolo

$$[\texttt{top}, \texttt{bottom}] \times [\texttt{left}, \texttt{right}]$$

dell'alignment graph, avente come sorgente il vertice $(\texttt{top}, \texttt{left})$ e come pozzo $(\texttt{bottom}, \texttt{right})$.

Tramite questa rappresentazione possiamo analizzare le direzioni possibili per gli archi all'interno del rettangolo. Un arco verticale da (i, j) a $(i+1, j)$ consuma il simbolo v_{i+1}. Un arco orizzontale da (i, j) a $(i, j+1)$ consuma w_{j+1}, mentre un arco diagonale consuma entrambi.

Calcolo degli score in spazio lineare. Il primo componente necessario è una versione di Needleman–Wunsch che, dato un rettangolo dell'alignment graph, restituisce gli score dei vertici appartenenti alla sua ultima colonna.

Come osservato precedentemente, per calcolare una colonna sono sufficienti la colonna precedente e i valori già determinati dalla colonna corrente. Indichiamo tali vettori rispettivamente con `previous` e `current`.

Nello pseudocodice seguente gli indici dei vettori coincidono, per semplicità, con gli indici delle righe dell'alignment graph.

Listing 3.1: Calcolo degli score in spazio lineare

```
NWScore(v,w,top,bottom,left,right,delta)
    previous[top] <- 0

    for i <- top+1,...,bottom
        previous[i] <- previous[i-1] + delta(v[i],'-')

    for j <- left+1,...,right
        current[top] <- previous[top] + delta('-',w[j])

        for i <- top+1,...,bottom
            vertical <- current[i-1] + delta(v[i],'-')
            horizontal <- previous[i] + delta('-',w[j])
            diagonal <- previous[i-1] + delta(v[i],w[j])
            current[i] <- max(vertical,horizontal,diagonal)

        swap(previous,current)

    return previous
```

Al termine della procedura,

$$\texttt{previous}[i]$$

rappresenta lo score del cammino ottimo dalla sorgente del sottoproblema $(\texttt{top}, \texttt{left})$ al vertice $(i, \texttt{right})$, per ogni $i \in \{\texttt{top}, \ldots, \texttt{bottom}\}$.

Possiamo notare che in particolare, quando $\texttt{right} = j_{\text{mid}}$, questo vettore contiene esattamente i valori che in precedenza chiamavamo prefix(i), relativi al sottoproblema corrente.

Per il calcolo dei valori suffix(i) utilizziamo una procedura simmetrica

$$\texttt{NWScoreReverse},$$

che visita il rettangolo a partire dal pozzo.

In questo caso

$$\texttt{previous}[i]$$

rappresenta lo score del cammino ottimo dal vertice $(i, \texttt{left})$ al pozzo del sottoproblema $(\texttt{bottom}, \texttt{right})$, per ogni $i \in \{\texttt{top}, \ldots, \texttt{bottom}\}$.

Listing 3.2: Calcolo inverso degli score

```
NWScoreReverse(v,w,top,bottom,left,right,delta)
    previous[bottom] <- 0

    for i <- bottom-1,...,top
        previous[i] <- previous[i+1] + delta(v[i+1],'-')

    for j <- right-1,...,left
        current[bottom] <- previous[bottom] + delta('-',w[j+1])

        for i <- bottom-1,...,top
            vertical <- current[i+1] + delta(v[i+1],'-')
            horizontal <- previous[i] + delta('-',w[j+1])
            diagonal <- previous[i+1] + delta(v[i+1],w[j+1])
            current[i] <- max(vertical,horizontal,diagonal)

        swap(previous,current)

    return previous
```

Determinazione del middle edge. Una volta individuato il middle vertex (i, j), rimane da scegliere l'arco ottimo uscente da esso.

Per effettuare il confronto sono necessari gli score a partire dalla colonna j e dalla colonna $j + 1$. Indichiamo con `suffix` e `suffix_next` i due vettori corrispondenti.

Listing 3.3: Determinazione del middle edge

```
MiddleEdge(v,w,bottom,mid_vertex,suffix,suffix_next,delta)
    i <- mid_vertex.i
    j <- mid_vertex.j

    if i = bottom
        return (i,j)->(i,j+1)

    vertical <- suffix[i+1] + delta(v[i+1],'-')
    horizontal <- suffix_next[i] + delta('-',w[j+1])
    diagonal <- suffix_next[i+1] + delta(v[i+1],w[j+1])
    best <- max(vertical,horizontal,diagonal)

    if best = diagonal
        return (i,j)->(i+1,j+1)

    else if best = vertical
        return (i,j)->(i+1,j)

    else
        return (i,j)->(i,j+1)
```

In caso di parità tra più valori candidati, è sufficiente scegliere uno qualunque degli archi che realizzano il massimo. Nel seguito adottiamo, per determinismo, la priorità diagonale, verticale, orizzontale.

Per individuare il middle edge sono necessari sia

$$R(i, j_{\text{mid}})$$

sia

$$R(i, j_{\text{mid}} + 1).$$

Entrambi possono essere ottenuti mediante `NWScoreReverse`, applicata rispettivamente alla colonna e a quella immediatamente successiva.

Procedura ricorsiva. Possiamo infine combinare le procedure precedenti.

I casi base verificano quando uno dei due lati del rettangolo ha lunghezza nulla. Se `left` = `right`, la sottostringa $v_{\texttt{top}+1} \cdots v_{\texttt{bottom}}$ viene allineata con la stringa vuota e l'allineamento contiene quindi soltanto archi verticali. Analogamente, se `top` = `bottom`, la sottostringa $w_{\texttt{left}+1} \cdot w_{\texttt{right}}$ viene allineata con la stringa vuota mediante soli archi orizzontali.

Negli altri casi viene individuato il middle edge e il problema viene risolto ricorsivamente nei due rettangoli che esso separa.

Listing 3.4: Algoritmo di Hirschberg

```
Hirschberg(v,w,top,bottom,left,right,delta)

    if left = right
        return allineamento verticale di v[top+1,...,bottom]

    if top = bottom
        return allineamento orizzontale di w[left+1,...,right]

    middle <- floor((left + right)/2)

    prefix <- NWScore(v,w,top,bottom,left,middle,delta)
    suffix <- NWScoreReverse(v,w,top,bottom,middle,right,delta)

    mid_i <- top

    for i <- top+1,...,bottom
        if prefix[i] + suffix[i] > prefix[mid_i] + suffix[mid_i]
            mid_i <- i

    mid_vertex <- (mid_i,middle)

    suffix_next <- NWScoreReverse(v,w,top,bottom,middle+1,right,delta)

    mid_edge <- MiddleEdge(v,w,bottom,mid_vertex,suffix,suffix_next,delta)

    left_alignment <- Hirschberg(v,w,top,mid_i,left,middle,delta)
    right_alignment <- Hirschberg(v,w,mid_edge.to_i,bottom,mid_edge.to_j,right
    ,delta)

    return concatenate(left_alignment,mid_edge,right_alignment)
```

Tramite un lieve abuso di notazione, nel listing la concatenazione con `mid_edge` indica l'aggiunta della singola colonna di allineamento determinata da tale arco.

3.3 Analisi della complessità

Studiamo adesso la complessità temporale e spaziale dell'algoritmo di Hirschberg.

Siano $n := |v|$ e $m := |w|$. Supponiamo, senza perdita di generalità, $n \leq m$, così da disporre la sequenza più corta lungo le righe dell'alignment graph.

Analizziamo innanzitutto il costo di un singolo sottoproblema. Come visto nella sezione precedente, esso corrisponde a un rettangolo dell'alignment graph; siano h e k rispettivamente la sua altezza e la sua larghezza.

La procedura `NWScore` visita ogni stato del rettangolo un numero costante di volte, richiedendo quindi tempo

$$\Theta(hk).$$

Essa mantiene solamente due vettori di $h + 1$ elementi, utilizzando quindi spazio

$$\Theta(h).$$

La stessa analisi vale per `NWScoreReverse`.

Per individuare il middle vertex e il middle edge viene eseguito un numero costante di scansioni del rettangolo. Sebbene ciò introduca un fattore costante aggiuntivo rispetto a una singola esecuzione di Needleman–Wunsch, il costo temporale complessivo associato a un rettangolo di dimensione $h \times k$ rimane

$$\Theta(hk).$$

Consideriamo adesso l'intera ricorsione. Dopo aver individuato il middle edge, il rettangolo corrente viene suddiviso in due sottoproblemi. Poiché la divisione avviene rispetto alla colonna centrale, l'area complessiva dei rettangoli processati diminuisce geometricamente procedendo nei livelli della ricorsione.

Sia $A_0 := nm$ l'area dell'alignment graph iniziale. Il lavoro complessivo può essere limitato, a meno di un fattore costante dovuto alle scansioni necessarie per ogni sottoproblema, dalla serie geometrica

$$A_0 + \frac{A_0}{2} + \frac{A_0}{4} + \cdots = \sum_{\ell=0}^{\infty} \frac{A_0}{2^\ell} = 2A_0.$$

L'uso del middle edge riduce leggermente l'area dei sottoproblemi rispetto alla variante basata sul solo middle vertex, senza modificare il limite asintotico. Nella presente

implementazione, inoltre, la determinazione dell'arco richiede il calcolo aggiuntivo di `suffix_next`; non è quindi possibile dedurre a priori un vantaggio temporale pratico dalla sola riduzione dell'area ricorsiva.

Ne segue che

$$T(n, m) = O(nm)$$

D'altra parte, già nella prima chiamata è necessario calcolare gli score relativi a una porzione complessivamente proporzionale all'intero alignment graph, richiedendo $\Omega(nm)$ operazioni. Allora

$$T(n, m) = \Theta(nm)$$

L'algoritmo di Hirschberg conserva dunque la complessità temporale asintotica rispetto all'algoritmo di Needleman–Wunsch. Il vantaggio riguarda l'utilizzo della memoria: la ricostruzione divide et impera richiede di ricalcolare alcuni score, introducendo un maggiore fattore costante nel tempo di esecuzione, ma evita di memorizzare l'intera matrice della programmazione dinamica.

Concentriamoci quindi adesso sulla complessità spaziale.

In un sottoproblema di altezza h, i vettori utilizzati dalle procedure `NWScore` e `NWScoreReverse` contengono $h + 1$ elementi. Poiché durante la determinazione del middle edge viene mantenuto un numero costante di tali vettori, la memoria richiesta dalle strutture di programmazione dinamica è

$$S_{\mathrm{DP}}(n, m) = \Theta\big(\min\{n, m\}\big).$$

I vettori utilizzati per individuare il middle edge sono necessari soltanto all'interno della chiamata corrente e possono essere liberati prima di procedere ricorsivamente con i due sottoproblemi. Conseguentemente, la memoria occupata dai vettori dei diversi livelli della ricorsione non si accumula quindi lungo lo stack.

Poiché a ogni livello la larghezza del sottoproblema viene circa dimezzata, la profondità della ricorsione è

$$O(\log m).$$

Ogni record di attivazione contiene soltanto un numero costante di indici e riferimenti, per cui lo stack ricorsivo richiede

$$S_{\mathrm{stack}}(n, m) = O(\log m).$$

Lo spazio ausiliario complessivo è pertanto

$$S_{\text{aux}}(n, m) = O(\min\{n, m\} + \log \max\{n, m\}).$$

In particolare, le strutture utilizzate dalla programmazione dinamica richiedono spazio lineare nella lunghezza della sequenza più corta. È in questo senso che Hirschberg viene definito un algoritmo di allineamento in spazio lineare.

Infine consideriamo la memoria utilizzata per salvare l'output. Come osservato per Needleman–Wunsch, se L indica il numero di colonne dell'allineamento finale, vale

$$\max\{n, m\} \leq L \leq n + m.$$

Poiché

$$\max\{n, m\} \geq \frac{n + m}{2},$$

segue che

$$L = \Theta(n + m).$$

La rappresentazione esplicita delle due sequenze allineate richiede quindi

$$S_{\text{out}}(n, m) = \Theta(n + m).$$

Se viene considerato anche l'output, lo spazio complessivo è pertanto

$$S_{\text{tot}}(n, m) = \Theta(n + m).$$

Abbiamo allora visto come Hirschberg conservi la stessa complessità temporale asintotica di Needleman–Wunsch, ma elimina la necessità di memorizzare le matrici complete degli score e dei predecessori. Lo spazio richiesto dalle strutture utilizzate durante il calcolo passa da

$$\Theta(nm)$$

a uno spazio lineare nella dimensione delle sequenze, e in particolare a

$$\Theta(\min\{n, m\})$$

per i vettori della programmazione dinamica.

3.4 Implementazione

L'implementazione di Hirschberg utilizza le stesse strutture dati e lo stesso modello di scoring introdotti per l'implementazione di Needleman–Wunsch. Quindi le sequenze vengono rappresentate dal tipo `Strand` mentre l'output viene restituito attraverso una struttura `Alignment`. Mancano ovviamente le matrici `score` e `pred`.

3.4.1 Calcolo degli score in spazio lineare

I sottoproblemi ricorsivi non vengono rappresentati creando nuove sottostringhe. Ciascuno di essi è identificato tramite quattro indici,

`v_source`, `v_sink`, `w_source`, `w_sink`,

che delimitano le porzioni delle due sequenze coinvolte, ricordando la rappresentazione degli indici del rettangolo per i sottoproblemi visti nelle sezioni precedenti.

La funzione `nw_score` implementa la procedura `NWScore` descritta nella Sezione 3.2. Quindi alloca due vettori di $n + 1$ interi, con $n =$ `v_sink` $-$ `v_source`.

Di seguito viene mostrato il frammento di codice che comprende l'inizializzazione dei due vettori e il calcolo degli score dell'ultima colonna del sottoproblema corrente.

```
previous[0] = 0;
for(size_t i=1; i<=n; i++) {
    previous[i] = previous[i-1] + scoring(v->bases[v_source + i - 1],BASE_GAP,model);
}
current[0] = 0;

for(size_t j=1; j<=m; j++) {
    current[0] = previous[0] + scoring(BASE_GAP,w->bases[w_source + j - 1],model);
    for(size_t i=1; i<=n; i++) {
        int up       = current[i-1]
            + scoring(v->bases[v_source + i - 1],BASE_GAP,model);
        int left     = previous[i]
            + scoring(BASE_GAP,w->bases[w_source + j - 1],model);
        int diagonal = previous[i-1]
            + scoring(v->bases[v_source + i - 1],
                      w->bases[w_source + j - 1],model);
        current[i] = max(up,left,diagonal);
    }
    int *temp = previous;
    previous  = current;
    current   = temp;
}
```

Gli indici `i` e `j` utilizzati all'interno della funzione sono relativi al sottoproblema corrente. Per accedere alla sequenza originale vengono quindi sommati gli offset `v_source` e `w_source`.

Terminato il calcolo, il vettore non più necessario viene liberato e `previous` viene restituito al chiamante.

La funzione `nw_score_reverse` realizza invece il calcolo simmetrico necessario per gli score di suffisso. In questo caso l'inizializzazione parte dall'ultima posizione del vettore e le sequenze vengono percorse in ordine decrescente, di seguito è possibile vederne il codice.

```
previous[n] = 0;
for(size_t i = n; i-- > 0; ) {
    previous[i] = previous[i+1] + scoring(v->bases[v_source + i], BASE_GAP,
    model);
}
```

È interessante anche vedere come è stata affrontata l'indicizzazione dei cicli nel verso contrario:

```
for(size_t j = m; j-- > 0; ) {
    [...]
    for(size_t i = n; i-- > 0; ) {
        [...]
    }
}
```

3.4.2 Middle vertex e middle edge

Per la ricerca del middle vertex vengono chiamate le funzioni per il calcolo degli score in tempo lineare, i cui risultati vengono salvati all'interno di due vettori, `prefix` e `suffix`.

```
size_t mid_j = w_source + m/2;

int *prefix = nw_score(v,v_source,v_sink,w,w_source,mid_j,model);
int *suffix = nw_score_reverse(v,v_source,v_sink,w,mid_j, w_sink,model);
```

Il seguente frammento mostra come viene ricercato l'elemento massimo di prefix(i) + suffix(i)

```
size_t mid_offset = 0;
int max = prefix[0] + suffix[0];
for(size_t i=1; i<=n; i++) {
    int score = prefix[i] + suffix[i];
    //length[i] = prefix[i] + suffix[i]
    if(max < score) {
        max = score;
        mid_offset = i;
    }
}
size_t mid_i = v_source + mid_offset;
```

Poiché i vettori sono indicizzati relativamente al sottoproblema, `mid_offset` identifica la riga del middle vertex rispetto a `v_source`.

Successivamente viene calcolato il vettore `suffix_next` tramite una chiamata a `nw_score_reverse`, alla quale viene passato l'indice `mid_j + 1`.

Il middle edge viene rappresentato da una struttura `MiddleEdge`, che memorizza il vertice iniziale, il vertice finale e la direzione dell'arco.

Di seguito è presente la funzione `mid_edge` che restituisce il middle edge di un sottoproblema.

```
MiddleEdge mid_edge(size_t top, size_t bottom,
                    size_t mid_vertex_i, size_t mid_vertex_j,
                    const int *suffix, const int* suffix_next,
                    const Strand *v, const Strand *w, const scoringModel *
model) {

    size_t i = mid_vertex_i - top;

    if(mid_vertex_i == bottom) {
        MiddleEdge mid_edge = {
            .from_i = mid_vertex_i,
            .from_j = mid_vertex_j,
            .to_i   = mid_vertex_i,
            .to_j   = mid_vertex_j + 1,
            .edge   = EDGE_HORIZONTAL
        };
        return mid_edge;
    }

    int next_down = suffix[i + 1] + scoring(v->bases[mid_vertex_i],BASE_GAP,
model);
    int next_right = suffix_next[i] + scoring(BASE_GAP,w->bases[mid_vertex_j],
model);
    int next_diagonal = suffix_next[i + 1] + scoring(v->bases[mid_vertex_i],w
->bases[mid_vertex_j],model);
    int final =  max(next_down,next_right,next_diagonal);

    MiddleEdge mid_edge = {
        .from_i = mid_vertex_i,
        .from_j = mid_vertex_j,
    };

    if(final == next_diagonal) {
        mid_edge.to_i = mid_vertex_i + 1;
        mid_edge.to_j = mid_vertex_j + 1;
        mid_edge.edge = EDGE_DIAGONAL;
    } else if(final == next_down) {
        mid_edge.to_i = mid_vertex_i + 1;
        mid_edge.to_j = mid_vertex_j;
        mid_edge.edge = EDGE_VERTICAL;
    } else {
        mid_edge.to_i = mid_vertex_i;
        mid_edge.to_j = mid_vertex_j + 1;
        mid_edge.edge = EDGE_HORIZONTAL;
    }

    return mid_edge;
}
```

Nella formulazione teorica l'arco uscente dal vertice (i, j) utilizza v_{i+1} e w_{j+1}. Gli array C sono indicizzati a partire da zero; dunque, se `mid_vertex_i` e `mid_vertex_j` rappresentano direttamente le coordinate del vertice, i simboli consumati dall'arco sono memorizzati in `v->bases[mid_vertex_i]` e `w->bases[mid_vertex_j]`.

3.4.3 Ricostruzione ricorsiva dell'allineamento

Avendo trovato middle vertex e middle edge siamo pronti per poter ricostruire l'allineamento.

Prima di analizzare la procedura ricorsiva citiamo velocemente i casi base, che sono stati implementati in maniera fedele al Listing 3.2; se una delle due sequenze è vuota, l'unico allineamento possibile consiste nell'allineare tutti i simboli dell'altra sequenza con gap.

Vediamo adesso la ricorrenza che concatena l'allineamento sinistro, il middle edge e l'allineamento destro.

```
free(prefix);
free(suffix);
free(suffix_next);

hirschberg_rec(v, v_source, mid_i, w, w_source, mid_j, model, alignment,
    length); // sinistra
emit(alignment, length, &edge, v, w, model);
hirschberg_rec(v, edge.to_i, v_sink, w, edge.to_j, w_sink, model, alignment,
    length); // destra
```

La ricostruzione dell'allineamento segue direttamente la decomposizione

$$A = A_{\text{left}} \circ e_{\text{mid}} \circ A_{\text{right}}$$

Prima di iniziare la ricorsione, la funzione pubblica `hirschberg` dispone lungo le righe la sequenza più corta. Se necessario, i due input vengono temporaneamente scambiati. In questo modo i vettori utilizzati dalle procedure di programmazione dinamica hanno sempre dimensione

$$\min\{n, m\} + 1,$$

realizzando concretamente il limite spaziale discusso nella Sezione 2.3.

Capitolo 4

Metodologia sperimentale

Dopo aver analizzato teoricamente e implementato gli algoritmi di Needleman-Wunsch e Hirschberg, possiamo procedere al loro confronto sperimentale.

L'obiettivo dell'analisi è verificare empiricamente le proprietà discusse nei capitoli precedenti, concentrandoci sul tempo di esecuzione e sul consumo di memoria al crescere della lunghezza delle sequenze.

Quando le risorse disponibili lo consentono, entrambi gli algoritmi vengono eseguiti sulla medesima istanza e con lo stesso modello di scoring. Per input la cui dimensione rende impraticabile l'allocazione delle matrici complete di Needleman–Wunsch, viene eseguito soltanto Hirschberg; tali casi vengono indicati esplicitamente nei risultati.

Nel seguito vengono descritti il dataset utilizzato, l'ambiente sul quale si è lavorato e il protocollo adottato per la raccolta delle misure, in modo tale da poter valutare la validità dell'esperimento, che verrà presentato nel capitolo successivo.

4.1 Dataset

Per il confronto sperimentale sono state utilizzate due classi di input: sequenze sintetiche generate in modo controllato e sequenze biologiche reali ottenute dal database NCBI Nucleotide [Nat].

Le sequenze sintetiche permettono di controllare direttamente le dimensioni n e m del problema e vengono quindi utilizzate principalmente per studiare la scalabilità dei due algoritmi. La generazione degli input viene effettuata mediante un generatore pseudocasuale deterministico, inizializzato attraverso un seed noto, così da rendere

riproducibili le istanze utilizzate. Sono state considerate tre famiglie di input sintetici:

`random` le due sequenze vengono generate indipendentemente sull'alfabeto $\Sigma_{\text{DNA}} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$;

`identiche` la seconda sequenza coincide con la prima;

`mutate` la seconda sequenza viene ottenuta dalla prima sostituendo ciascun simbolo con probabilità 0.05. In caso di sostituzione viene scelta una delle altre tre basi.

Sono state considerate sequenze di lunghezza crescente per generare input quadrati ($n = m$) di lunghezza

$$250,\ 500,\ 1000,\ 2000,\ 4000,\ 8000,\ 12000.$$

Per ogni combinazione di dimensione e famiglia vengono generate sette istanze indipendenti per configurazione, ottenute mediante seed differenti. Per ciascun seed entrambi gli algoritmi vengono quindi eseguiti esattamente sulla stessa coppia di sequenze.

Sono inoltre considerate alcune istanze "rettangolari", con $n \neq m$, al fine di osservare il comportamento degli algoritmi anche quando le due sequenze hanno lunghezze significativamente differenti.

Agli input sintetici sono affiancate quattro coppie di sequenze biologiche reali, scelte in modo da coprire dimensioni progressivamente maggiori.

Dataset	**Sequenza 1**	**Sequenza 2**	**Lunghezza**
RNA ribosomiale batterico	*E. coli* `NR_112059.1`	*S. enterica* `NR_116005.1`	$\sim 1.5 \cdot 10^3$
HIV-1	HXB2 (subtipo B) `NC_001802.1`	ETH2220 (subtipo C) `U46016.1`	$\sim 10^4$
SARS-CoV-2	Wuhan-Hu-1 `NC_045512.2`	Delta (B.1.617.2) `OK091006.1`	$\sim 3 \cdot 10^4$
Genomi plastidici	*A. thaliana* `NC_000932.1`	*O. sativa* `NC_001320.1`	$\sim 1.5 \cdot 10^5$

Tabella 4.1: Coppie di sequenze biologiche reali utilizzate nel confronto sperimentale.

Le sequenze sono state scaricate manualmente in formato FASTA da NCBI Nucleotide utilizzando gli accession number indicati.

Nella prima coppia sono presenti anche dei simboli `N`, mentre nella sequenza Delta (B.1.617.2) è presente il simbolo `Y`, che indicano nucleotidi non determinati, questi vengono sostituiti preliminarmente con simboli scelti pseudocasualmente da Σ_{DNA}. Il preprocessing utilizza un seed fissato e viene eseguito una sola volta prima della raccolta delle misure, così da garantire che entrambi gli algoritmi operino sulle medesime sequenze e che l'esperimento sia riproducibile. Le sequenze reali sono utilizzate come carichi di lavoro realistici di dimensione crescente. Lo score ottenuto non viene pertanto interpretato dal punto di vista biologico; l'obiettivo del loro utilizzo è valutare sperimentalmente tempo di esecuzione e consumo di memoria dei due algoritmi.

4.2 Ambiente hardware e software

Tutti gli esperimenti sono stati eseguiti sul medesimo sistema, in modo da mantenere costanti le principali condizioni che possono influenzare le prestazioni.

Componente	**Configurazione**
Processore	Snapdragon(R) X 12-core X1E80100 @ 3.40 GHz
Architettura	ARM64 (AArch64)
Memoria RAM	16,0 GB
Memoria visibile a WSL 2	9.7 GiB
Swap WSL 2	4.0 GiB
Sistema operativo host	Microsoft Windows 11
Ambiente di esecuzione	WSL 2 (2.6.1.0) – Ubuntu 24.04.5 LTS
Compilatore	GCC
Versione compilatore	13.3.0
Flag di compilazione	`-O2 -Wall -Wextra -Wpedantic -std=c99`

Tabella 4.2: Ambiente utilizzato per gli esperimenti.

Sebbene il sistema host disponga di 16 GB di memoria fisica, l'ambiente WSL 2 utilizzato per gli esperimenti dispone di circa 9.7 GiB di memoria RAM, oltre a 4.0 GiB di spazio di swap. Tali valori costituiscono quindi il riferimento effettivo per la valutazione della fattibilità delle esecuzioni.

4.3 Misura dei tempi e della memoria

Ogni esecuzione del benchmark avviene in un processo separato. La misura temporale comprende esclusivamente l'esecuzione dell'algoritmo di allineamento. La generazione di input sintetici, la lettura e il parsing dei file FASTA e le successive verifiche di correttezza vengono pertanto effettuati al di fuori della regione temporizzata.

Vengono raccolte due misure temporali. Il tempo reale trascorso, o *wall time*, viene misurato utilizzando `CLOCK_MONOTONIC`, mentre il tempo di CPU viene ottenuto mediante `CLOCK_PROCESS_CPUTIME_ID`. Il wall time viene utilizzato come misura principale delle prestazioni temporali, mentre il CPU time costituisce una misura complementare.

Per gli input sintetici, l'ordine delle esecuzioni di Needleman–Wunsch e Hirschberg viene alternato tra istanze successive, così da evitare di favorire sistematicamente uno dei due algoritmi a causa dell'ordine di esecuzione.

Per le sequenze biologiche reali la coppia di input rimane invece invariata tra le diverse esecuzioni. Anche in questo caso viene alternato l'ordine dei due algoritmi quando entrambi possono essere eseguiti.

Il consumo di memoria viene misurato tramite il massimo *resident set size* del processo, ottenuto mediante `/usr/bin/time`. La misura, espressa in KiB, rappresenta il massimo quantitativo di memoria residente raggiunto durante l'intera esecuzione del processo e comprende quindi, oltre alle strutture ausiliarie dell'algoritmo, anche input, output, runtime e altro overhead del processo. Essa non coincide pertanto con lo spazio ausiliario teorico analizzato nei capitoli precedenti, ma consente di confrontare empiricamente la diversa crescita della memoria richiesta dalle due implementazioni eseguite nelle medesime condizioni.

Al termine viene verificata la validità strutturale dell'allineamento prodotto. Eliminando i gap dalle due sequenze allineate devono essere ricostruiti esattamente gli input originali; inoltre lo score dell'allineamento viene ricalcolato e confrontato con quello restituito dall'algoritmo. Quando una stessa istanza viene elaborata da entrambi gli algoritmi, viene verificato inoltre che Needleman–Wunsch e Hirschberg restituiscano lo stesso score ottimo.

Le esecuzioni terminate correttamente vengono aggregate per configurazione. Per wall time, CPU time e memoria vengono calcolate la mediana, media e deviazione standard. La mediana viene usata come indicatore principale, poiché meno sensibile a eventuali esecuzioni anomale. Le esecuzioni fallite o gli eventuali esaurimenti di memoria vengono invece registrati separatamente e non vengono inclusi nelle statistiche delle esecuzioni valide.

Capitolo 5

Risultati e confronto

In questo capitolo vengono presentati e discussi i risultati ottenuti mediante il protocollo sperimentale descritto nel Capitolo 4. L'analisi ha due obiettivi principali: verificare empiricamente la crescita del tempo di esecuzione dei due algoritmi e confrontarne il consumo di memoria.

Salvo diversa indicazione, vengono considerate le mediane delle sette esecuzioni valide per ciascuna configurazione.

5.1 Validazione dei risultati

Verifichiamo innanzitutto la correttezza dei risultati degli algoritmi.

Indicatore	**Risultato**
Run complessive	406
Esecuzioni completate	399
Esecuzioni NW non avviate per limite di memoria	7
Coppie NW/Hirschberg confrontabili	196
Coppie con score coincidente	196
Allineamenti strutturalmente invalidi	0

Tabella 5.1: Validità risultati

Tutte le esecuzioni completate hanno prodotto un allineamento strutturalmente valido. Per le 196 istanze elaborate da entrambi gli algoritmi, Needleman–Wunsch e Hirschberg hanno inoltre restituito lo stesso score ottimo. Le esecuzioni non confron-

tabili sono quelle sui genomi plastidici, queste vengono trattate separatamente e non costituiscono un errore nell'esperimento, ma un risultato atteso.

5.2 Scalabilità su sequenze sintetiche

Per valutare sperimentalmente la crescita del costo computazionale sono state considerate innanzitutto le istanze sintetiche quadrate, con

$$n = m \in \{250, 500, 1000, 2000, 4000, 8000, 12000\}.$$

Le tre famiglie di input, `random`, `identiche`, `mutate`, presentano lo stesso andamento generale; le differenze nel contenuto delle sequenze influenzano i fattori costanti del tempo di esecuzione, ma non modificano la quantità di stati della programmazione dinamica che devono essere elaborati.

Tempo di esecuzione. La Figura 5.1 riporta, in scala logaritmica, il wall time mediano al crescere della lunghezza delle sequenze.

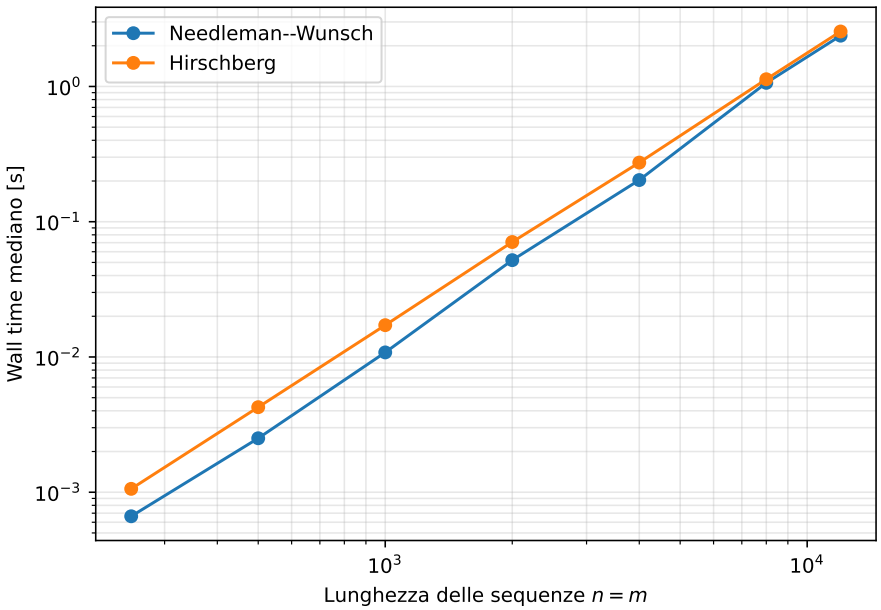


Figura 5.1: Wall time mediano di Needleman–Wunsch e Hirschberg sulle istanze sintetiche `random` con $n = m$. Entrambi gli assi sono rappresentati in scala logaritmica.

Per le sequenze `random`, Needleman–Wunsch passa da un tempo mediano di circa $6.64 \cdot 10^{-4}$s per $n = 250$ a 2.374s per $n = 12000$. Nello stesso intervallo Hirschberg passa da $1.06 \cdot 10^{-3}$s a 2.549s.

L'aumento osservato è quindi compatibile, per entrambi gli algoritmi, con la crescita quadratica prevista dall'analisi teorica,

$$T(n, n) = \Theta(n^2).$$

Una regressione lineare dei valori in scala logaritmica restituisce una pendenza pari circa a 2.13 per Needleman–Wunsch e 2.01 per Hirschberg. Tali valori sono prossimi all'esponente 2 previsto dalla complessità $\Theta(n^2)$.

La stessa tendenza emerge anche per le altre famiglie di input. Per esempio, per $n = 12000$ i tempi mediani risultano

Tipo di input	**Needleman–Wunsch**	**Hirschberg**
`random`	2.374s	2.549s
`mutate`	2.127s	2.056s
`identiche`	2.049s	2.169s

Tabella 5.2: Wall time mediano per $n = 12000$

Le differenze tra le tre famiglie rimangono quindi contenute rispetto alla crescita determinata dalla dimensione dell'input.

Per le dimensioni più piccole Hirschberg presenta generalmente un overhead maggiore. Nel caso `random`, per esempio, a $n = 1000$ vengono misurati 0.0108s per Needleman–Wunsch e 0.0172s per Hirschberg. All'aumentare della dimensione, tale differenza relativa si riduce: a $n = 8000$ i tempi mediani sono rispettivamente 1.063s e 1.131s.

Questo comportamento è coerente con il fatto che Hirschberg mantiene la stessa complessità temporale asintotica di Needleman–Wunsch, pur introducendo lavoro aggiuntivo per il calcolo degli score necessari alla decomposizione ricorsiva. Tale costo aggiuntivo non determina, negli esperimenti effettuati, una separazione sostanziale dei tempi per gli input di dimensione maggiore.

Consumo di memoria. La differenza più marcata tra le due implementazioni riguarda, ovviamente, il consumo di memoria. La Figura 5.2 riporta il massimo RSS mediano sulle stesse istanze sintetiche.

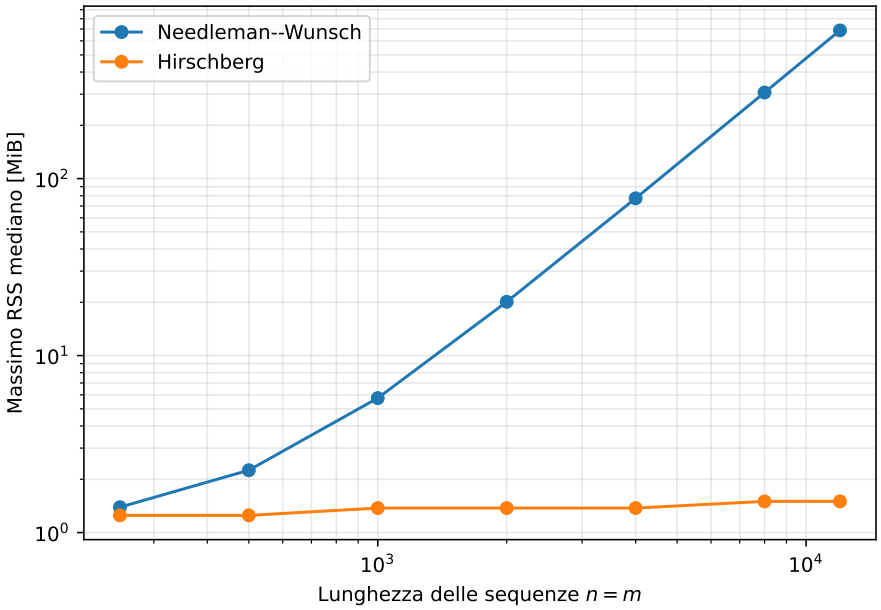


Figura 5.2: Massimo resident set size mediano di Needleman–Wunsch e Hirschberg sulle istanze sintetiche `random` con $n = m$. Entrambi gli assi sono rappresentati in scala logaritmica.

Considerando ancora le istanze `random`, il massimo resident set size mediano di Needleman–Wunsch cresce come segue:

n	RSS_{NW}
1000	5.75 MiB
2000	20.13 MiB
4000	77.38 MiB
8000	306.38 MiB
12000	687.88 MiB

Per valori sufficientemente grandi, il raddoppio della lunghezza produce quindi un aumento della memoria vicino a un fattore quattro, in accordo con la dipendenza quadratica

$$S_{\mathrm{NW}}(n, n) = \Theta(n^2).$$

Considerando le dimensioni $n \geq 1000$, una regressione in scala logaritmica restituisce per Needleman–Wunsch una pendenza di circa 1.93, coerente con la crescita quadratica attesa.

Hirschberg mostra invece un comportamento completamente diverso. Sulle stesse istanze `random`, il massimo RSS mediano passa da circa 1.25 MiB per $n = 250$ a solamente 1.50 MiB per $n = 12000$.

In particolare, osserviamo che per l'istanza 12000×12000 vengono misurati

$$687.88\text{MiB}$$

per Needleman–Wunsch, mentre soltanto

$$1.50\text{MiB}$$

per Hirschberg, con un rapporto di circa

$$\frac{687.88}{1.50} \simeq 459.$$

Ricordiamo che la misura RSS non coincide tuttavia con lo spazio ausiliario teorico dell'algoritmo, poiché comprende anche input, output, runtime e altre strutture del processo, come già discusso nella Sezione 4.

Di conseguenza, dai valori costanti osservati per Hirschberg non segue che il suo utilizzo di memoria sia costante. La componente teoricamente lineare,

$$\Theta(\min\{n, m\}),$$

rimane relativamente piccola rispetto all'overhead di base del processo nell'intervallo di dimensioni considerato e la sua crescita viene quindi parzialmente mascherata dalla misura complessiva di RSS.

Istanze rettangolari. Le istanze rettangolari permettono inoltre di verificare il comportamento quando le lunghezze delle due sequenze sono significativamente differenti.

Per la coppia di dimensioni 1000×10000 e 10000×1000, Hirschberg utilizza in entrambi i casi un RSS mediano pari a 1.375 MiB; lo stesso valore rimane anche per 2000×20000 e 20000×2000.

Questo risultato è coerente con la scelta implementativa di disporre la sequenza più corta lungo la dimensione dei vettori utilizzati per il calcolo degli score, in modo che la memoria ausiliaria dipenda da

$$\min\{n, m\}.$$

Needleman–Wunsch, al contrario, deve memorizzare strutture di taglia proporzionale al prodotto $n \times m$. Il massimo RSS risulta infatti 48.75 MiB per entrambe le orientazioni della coppia 1000×10000 e 191.88 MiB per entrambe le orientazioni della coppia 2000×20000.

5.3 Risultati sulle sequenze biologiche reali

Gli esperimenti sulle sequenze biologiche reali consentono di osservare lo stesso confronto su input di dimensioni progressivamente maggiori. Come specificato nel Capitolo 4, tali sequenze vengono utilizzate come carichi di lavoro realistici e gli score ottenuti non vengono interpretati dal punto di vista biologico.

I risultati principali sono riportati nelle Tabelle 5.3 e 5.4.

Dataset	**Dimensioni**	**Wall time NW**	**Wall time Hirschberg**
16S rRNA	1514×1445	0.025s	0.034s
HIV-1	9181×9031	1.194s	1.381s
SARS-CoV-2	29903×29836	12.233s	12.951s
Plastidi	154478×134525	–	327.265s

Tabella 5.3: Wall time sequenze biologiche

Notiamo, come ci aspettavamo, che anche sulle sequenze reali i tempi dei due algoritmi rimangono dello stesso ordine di grandezza.

La differenza relativa diminuisce al crescere della dimensione delle sequenze, senza considerare le sequenze per le quali Needleman–Wunsch non ha potuto allocare memoria. Per esempio, vediamo come Hirschberg richieda circa il 5.9% di tempo in più rispetto a Needleman–Wunsch sul dataset SARS-CoV-2.

Dataset	**Dimensioni**	**RSS NW**	**RSS Hirschberg**
16S rRNA	1514×1445	11.50 MiB	1.38 MiB
HIV-1	9181×9031	396.50 MiB	1.50 MiB
SARS-CoV-2	29903×29836	4255.75 MiB	1.68 MiB
Plastidi	154478×134525	–	3.52 MiB

Tabella 5.4: RSS sequenze biologiche

La differenza nel consumo di memoria è invece molto più pronunciata. Per HIV-1 vengono misurati 396.50 MiB per Needleman–Wunsch contro 1.50 MiB per Hirschberg,

con un rapporto superiore a 264.

Nel caso SARS-CoV-2, invece si raggiunge un rapporto di

$$\frac{4255.75}{1.684} \simeq 2528.$$

Pertanto, su questa istanza, una differenza temporale inferiore al 6% è accompagnata da una riduzione del massimo RSS superiore a tre ordini di grandezza.

Il caso dei genomi plastidici. La coppia di genomi plastidici costituisce l'istanza di dimensione maggiore, con

$$n = 154478, \qquad m = 134525.$$

Hirschberg completa tutte le sette esecuzioni con un wall time mediano di

$$327.265 \text{ s}$$

e un massimo RSS mediano di

$$3.52 \text{ MiB}.$$

Per Needleman–Wunsch, invece, la dimensione delle matrici previste dall'implementazione porta a una stima della memoria richiesta di circa 96.8 GiB, largamente superiore alle risorse rese disponibili all'ambiente WSL utilizzato per gli esperimenti, Tabella 4.2.

Il caso plastidico mostra quindi la conseguenza pratica della diversa complessità spaziale: la riduzione della memoria non costituisce soltanto un miglioramento quantitativo, ma permette a Hirschberg di affrontare, sul medesimo ambiente hardware, istanze per le quali la rappresentazione completa utilizzata da Needleman–Wunsch richiederebbe una quantità di memoria non disponibile.

5.4 Conclusioni, Needleman–Wunsch vs Hirschberg

I risultati sperimentali risultano complessivamente coerenti con l'analisi teorica sviluppata nei capitoli precedenti.

Per quanto riguarda il tempo di esecuzione, sia Needleman–Wunsch che Hirschberg mostrano sulle istanze sintetiche una crescita compatibile con

$$\Theta(nm),$$

Hirschberg introduce un costo aggiuntivo dovuto al ricalcolo degli score necessario alla decomposizione divide et impera, ma tale overhead risulta contenuto negli esperimenti effettuati e tende a diventare relativamente meno rilevante per le istanze di dimensione maggiore.

La differenza principale emerge invece nell'utilizzo della memoria. Needleman–Wunsch mantiene esplicitamente le matrici necessarie al calcolo e alla ricostruzione dell'allineamento e presenta quindi una crescita coerente con

$$\Theta(nm).$$

Hirschberg evita la memorizzazione dell'intera matrice e utilizza strutture di programmazione dinamica di dimensione lineare nella sequenza più corta. Nei test sintetici ciò porta, per $n = 12000$, da circa 688 MiB a 1.5 MiB di massimo RSS; sul dataset SARS-CoV-2 si passa da circa 4.16 GiB a 1.68 MiB.

È stato quindi verificato il vantaggio dell'algoritmo di Hirschberg nella possibilità di mantenere un tempo di esecuzione dello stesso ordine di Needleman–Wunsch riducendo drasticamente la memoria necessaria.

Tale differenza diventa particolarmente significativa all'aumentare della dimensione delle sequenze: mentre per input piccoli entrambe le implementazioni risultano facilmente utilizzabili, per input più grandi la memoria richiesta dalle matrici di Needleman–Wunsch diventa progressivamente il principale limite pratico. Hirschberg estende quindi l'insieme delle istanze elaborabili sulle medesime risorse hardware mantenendo la ricostruzione completa di un allineamento ottimo.

Bibliografia

[CLRS22] Thomas H Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, 4 edition, 2022.

[CP15] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms : An Active Learning Approach*, volume 1. Active Learning Publishers, 2 edition, 2015.

[Hir75] Daniel S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975.

[JP04] Neil C. Jones and Pavel A. Pevzner. *An Introduction to Bioinformatics Algorithms*. The MIT Press, Cambridge, Massachusetts, 2004.

[Nat] National Center of Biotechnology Information. NCBI Nucleotide. `https://www.ncbi.nlm.nih.gov/nucleotide`.

[NW70] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.